Fire Extinguisher Management System

Activity 1 – Requirement Analysis & Design

Microservices Architecture | RESTful API Design

Prepared for: TZW LTD
Role: Full Stack Developer
Date: June 2026

Table of Contents

1. Introduction and Context

TZW LTD is a company dedicated to managing, inspecting, and maintaining fire safety equipment — particularly fire extinguishers — across large commercial and industrial facilities. Their existing Fire Extinguisher Management System is built on a monolithic architecture and is now facing several operational challenges, including missed inspection deadlines, difficulty tracking maintenance history, and compliance issues.

To resolve these problems, TZW LTD seeks to upgrade the system to a microservices architecture based on RESTful principles. The new system must allow users to check extinguisher statuses, schedule inspections, log maintenance actions, track compliance, and generate real-time reports, while ensuring scalability, maintainability, and high availability.

This document covers Activity 1 of the engagement: identifying and defining the required microservices and their REST API contracts, defining and designing the database model, and describing the user-registration / signup interface mockup.

2. From Monolith to Microservices

A monolithic system packages every concern — users, extinguishers, inspections, and reporting — into a single deployable unit that shares one database. This makes the system hard to scale selectively, risky to deploy (one change can break everything), and difficult to maintain as it grows.

A microservices architecture decomposes the system into small, independently deployable services, each owning a single business capability and its own database. Services communicate only through well-defined REST APIs over HTTP. The chosen benefits for TZW LTD are:

- Scalability: the reporting service can be scaled independently during heavy reporting periods without scaling the rest of the system.
- Maintainability: each service has a small, focused codebase that a team can understand and change in isolation.
- High availability & fault isolation: if one service fails (for example, reporting), the others (registration, inspections) keep working.
- Independent deployment: services can be released on their own schedules, reducing deployment risk.

3. Identified Microservices

Analysing the business requirements, the system is decomposed into four core domain services plus an API gateway that acts as the single entry point for all clients.

Service	Responsibility	Owns Data
User Service	Authentication, roles (Admin/Inspector/User), registration, JWT issuing, profile and password management.	Users, reset tokens
Extinguisher Service	Register, list, view, update and remove extinguishers; log maintenance activities.	Extinguishers, maintenance logs
Inspection Service	Schedule inspections (select extinguisher, choose	Inspections,

Service	Responsibility	Owns Data
	date/time), notify personnel, track inspection status and results.	notifications
Reporting Service	Aggregate data from other services to produce real-time daily / monthly / yearly reports (stock counts, inspection status).	None (read-only aggregator)
API Gateway	Single public entry point; routes requests to the correct internal service and centralises cross-cutting concerns.	None

Each domain service follows the database-per-service pattern: it owns its schema and never reaches into another service's database. Cross-service needs (for example, the inspection service confirming an extinguisher exists, or the reporting service counting stock) are satisfied through HTTP calls to the owning service's REST API.

4. RESTful API Contract

The API follows REST conventions: resources are nouns, HTTP verbs express the action, and status codes communicate the outcome. Authentication uses a JSON Web Token (JWT) supplied in the Authorization: Bearer header. The full machine-readable contract is published per service as OpenAPI/Swagger documentation at each service's /api/docs endpoint.

4.1 User Service Endpoints

Method	Path	Description	Access
POST	/api/auth/register	Register a new user (signup)	Public
POST	/api/auth/login	Authenticate and receive a JWT	Public
POST	/api/auth/logout	Revoke the current token	Authenticated
POST	/api/auth/forgot-password	Request a password reset token	Public
POST	/api/auth/reset-password	Reset password using a token	Public
GET	/api/users/me	View own profile	Authenticated
PUT	/api/users/me	Update own profile	Authenticated
PUT	/api/users/me/password	Change password	Authenticated
GET	/api/users	List all users	Admin
POST	/api/users	Create a user with any role	Admin
PATCH	/api/users/{id}	Update a user's role / status	Admin

4.2 Extinguisher Service Endpoints

Method	Path	Description	Access
POST	/api/extinguishers	Register a new extinguisher	Admin, Inspector
GET	/api/extinguishers	List all extinguishers (filterable)	Authenticated
GET	/api/extinguishers/{id}	View extinguisher details by id	Authenticated
PUT	/api/extinguishers/{id}	Update extinguisher information	Admin, Inspector
DELETE	/api/extinguishers/{id}	Remove an extinguisher record	Admin
POST	/api/extinguishers/{id}/maintenance	Log a maintenance activity	Admin, Inspector
GET	/api/extinguishers/{id}/maintenance	List maintenance logs	Authenticated

4.3 Inspection Service Endpoints

Method	Path	Description	Access
POST	/api/inspections	Schedule an inspection and notify personnel	Admin, Inspector
GET	/api/inspections	List inspections (filter by status/upcoming)	Authenticated
GET	/api/inspections/{id}	View an inspection and its notifications	Authenticated
PUT	/api/inspections/{id}	Update / complete an inspection	Admin, Inspector
DELETE	/api/inspections/{id}	Cancel / delete an inspection	Admin

4.4 Reporting Service Endpoints

Method	Path	Description	Access
GET	/api/reports	Summary report across all data	Admin, Inspector
GET	/api/reports/{period}	Report for daily / monthly / yearly	Admin, Inspector
GET	/api/reports/stream/live	Real-time report stream (SSE)	Admin, Inspector

5. Database Model

Following the database-per-service principle, each service defines its own schema. The logical model below shows every table, its key columns, and the relationships within each service. Cross-service references (such as an inspection pointing at an extinguisher) are stored as plain identifiers, not foreign keys, because the data lives in a different service's database.

5.1 User Service – users

Column	Type	Notes
id	TEXT (UUID)	Primary key
first_name	TEXT	Required
last_name	TEXT	Required
email	TEXT	Required, unique
password_hash	TEXT	bcrypt hash, never stored in plain text
role	TEXT	Admin | Inspector | User (enforced by CHECK)
is_active	INTEGER	Soft enable/disable flag
reset_token / reset_expires	TEXT / INTEGER	Password recovery
created_at / updated_at	TEXT	Audit timestamps

5.2 Extinguisher Service – extinguishers & maintenance_logs

extinguishers holds the core asset record required by the brief: Serial Number, Location, Type, Size, Installation Date, Expiry date and Status.

Column	Type	Notes
id	TEXT (UUID)	Primary key
serial_number	TEXT	Required, unique
location	TEXT	Physical location
type	TEXT	Water | CO2 | Foam | Dry Chemical
size	TEXT	2.5 / 5 / 9 / 12 lbs
installation_date	TEXT	Date installed
expiry_date	TEXT	Date of expiry
status	TEXT	Active | Expired | Needs Maintenance | Decommissioned

maintenance_logs records each maintenance activity (Activity 3g): action taken, date of the action, relevant personnel, and conditions noted. It references extinguishers via a foreign key (same service, same database).

Column	Type	Notes
id	TEXT (UUID)	Primary key
extinguisher_id	TEXT	FK → extinguishers.id (CASCADE)
action_taken	TEXT	What was done
action_date	TEXT	Date of the action
personnel	TEXT	Who performed it
conditions_noted	TEXT	Conditions observed during maintenance

5.3 Inspection Service – inspections & notifications

inspections captures the scheduling requirements of Activity 3f: the selected extinguisher, the chosen date/time, the assigned personnel, and the resulting status. notifications stores the messages sent to relevant personnel when an inspection is scheduled.

Column	Type	Notes
id	TEXT (UUID)	Primary key
extinguisher_id	TEXT	Identifier of the selected extinguisher
serial_number	TEXT	Denormalised for convenience
scheduled_at	TEXT	Chosen date and time
assigned_to	TEXT	Relevant personnel notified
status	TEXT	Scheduled | Completed | Cancelled | Missed
result / notes	TEXT	Outcome of the inspection
created_by	TEXT	Who scheduled it

5.4 Reporting Service

The reporting service owns no tables. It is a read-only aggregator that calls the extinguisher and inspection services over HTTP and composes the totals (stock count, breakdown by status/type, and inspection status) on demand, including a real-time Server-Sent Events stream.

6. User Roles and Access Control

Three roles are defined, each with distinct privileges enforced by role-based access control (RBAC) on every protected endpoint.

Role	Capabilities
Admin	Manages overall system features, user accounts and data integrity. Full access including creating users, assigning roles and removing extinguisher records.
Inspector	Responsible for conducting inspections, logging results and scheduling maintenance. Can register and update extinguishers and log maintenance.
User	Can view extinguisher status and schedule inspections (read-mostly). Cannot delete records or manage other users.

7. Security and Authentication

Security spans the whole system and is implemented as follows:

1. Passwords are hashed with bcrypt and never stored or returned in plain text.
2. Authentication is stateless and token-based using JWT. On login the user service signs a token carrying the user id, email and role; clients send it on every subsequent request.
3. Authorization is role-based: each protected route declares the roles allowed to call it, and a shared middleware rejects requests from users who lack the required role (HTTP 403).
4. Logout revokes the active token via a server-side blocklist so it can no longer be used.
5. Password recovery issues a short-lived, single-use reset token rather than exposing or emailing the password.

8. User Registration / Signup Interface

The signup form is the primary entry point for new users and collects exactly the fields defined in the brief: First Name, Last Name, Email and Password. The intended layout (designed in Figma) is described below; a wireframe is included in the project as figma-signup-wireframe.svg.

- A centred card titled “Create your TZW account” on a clean, neutral background.
- Stacked input fields in order: First Name, Last Name, Email, Password (masked, with a show/hide toggle and a strength hint).
- Inline validation: email format and password strength (minimum 8 characters including a letter and a number) are checked before the form can be submitted.
- A primary “Create account” button, with a secondary “Already have an account? Sign in” link below.
- On success the user is registered with the default “User” role and redirected to the login screen.

The form maps directly onto the POST /api/auth/register endpoint, and the validation rules mirror the server-side validation so users get immediate feedback while the server remains the source of truth.

9. Summary

The proposed design replaces TZW LTD's monolith with four focused, independently deployable microservices behind a single API gateway, each owning its own data and exposing a documented RESTful API secured with JWT and role-based access control. The database model captures all required records for users, extinguishers, maintenance, inspections and notifications, and the reporting service delivers the real-time daily, monthly and yearly reports the business needs — together satisfying the scalability, maintainability and high-availability goals set out in the brief.